

Deploy Manager - Documentación Técnica

Deploy Manager — Documentación Técnica

Requisitos del sistema

Servidor del panel

- **PHP 7.4+** con PHP-FPM (`fastcgi_finish_request()` para ejecución en background)
- **`exec()` habilitado** — no puede estar en `disable_functions` del `php.ini`
- **Extensión `ZipArchive`** habilitada
- **Binarios:** `ssh`, `scp`, `rsync`, `tar` accesibles por el usuario web (`www-data`, `apache`, etc.)
- **Dolibarr 16+** funcional con permisos de escritura en `DOL_DATA_ROOT`
- **Clave SSH `ed25519`** dedicada con permisos `600`, legible por el proceso PHP

Servidores destino

- Clave pública del panel en `~/.ssh/authorized_keys` del usuario SSH configurado
- Usuario SSH con `sudo` sin contraseña para: `rsync`, `tar`, `chown`, `mysql`, `find`
- Si se usa `sudoers` restringido:

```
# /etc/sudoers.d/deployer
deployer ALL=(ALL) NOPASSWD: /usr/bin/rsync, /bin/tar, /bin/
chown, /usr/bin/mysql, /usr/bin/find
```

- `rsync` y cliente `mysql/mariadb` instalados
- `git` configurado con `remote origin` si las instancias usan repositorios

Verificación rápida

```
# En el servidor del panel, verificar binarios:
which ssh scp rsync tar
```

```
# Verificar exec() habilitado:
```

```

php -r "exec('echo OK', \$o); echo implode('', \$o);"

# Verificar ZipArchive:
php -r "echo class_exists('ZipArchive') ? 'OK' : 'FALTA';"

# Verificar PHP-FPM:
php -r "echo function_exists('fastcgi_finish_request') ? 'OK' :
        'No (mod_php)';"

# Verificar conexión SSH a servidor destino:
ssh -i /ruta/clave -o BatchMode=yes usuario@ip echo OK

```

Arquitectura

Estructura de archivos

```

htdocs/custom/deploymanager/
├── core/modules/
│   └── modDeployManager.class.php    # Descriptor del módulo
├── class/
│   ├── sshexecutor.class.php        # Wrapper SSH/rsync/SCP
│   └── deployengine.class.php       # Orquestador de
despliegues
│   └── modulescanner.class.php       # Escaneo de módulos en
instancias
│   └── modulepackager.class.php      # Validación y
almacenamiento de ZIPs
├── ajax/
│   ├── test_connection.php          # Test SSH a servidor
│   └── scan_instance.php            # Escanear módulos de
instancia
│   └── autodiscover.php              # Descubrir instancias en
servidor
│   ├── upload_release.php           # Subir ZIP de módulo
│   ├── deploy_execute.php           # Lanzar despliegue
│   └── deploy_status.php            # Consultar estado de batch
├── scripts/
│   └── deploy_worker.php             # Worker CLI (no usado
actualmente)
├── lib/
│   └── deploymanager.lib.php         # Funciones helper
├── css/
│   └── deploymanager.css
├── js/
│   └── deploymanager.js
├── langs/es_ES/
│   └── deploymanager.lang
├── dashboard.php                    # Panel principal
├── server_list.php / server_card.php # CRUD servidores
├── instance_list.php / instance_card.php # CRUD instancias
└── module_list.php / module_card.php # Lista/detalle módulos

```

└─ release_list.php	# Subir/listar releases
└─ deploy_wizard.php	# Wizard de despliegue
└─ deploy_status.php	# Progreso en tiempo real
└─ deploy_history.php	# Historial

Flujo de datos

Usuario → deploy_wizard.php → ajax/deploy_execute.php → DeployEngine

SSHExecutor (backup)

rsyncBetweenServers (copia archivos)

SSHExecutor (git sync)

SSHExecutor (SQL)

SSHExecutor (verificar versión)

Base de datos

Todas las tablas tienen prefijo llx_deploymanager_. Entity = 1.

llx_deploymanager_server

Campo	Tipo	Descripción
rowid	INT AI PK	ID
name	VARCHAR(100)	Nombre identificativo
host	VARCHAR(255)	IP o hostname
ssh_user	VARCHAR(50)	Usuario SSH (default: deployer)
ssh_port	INT	Puerto SSH (default: 22)
ssh_key_path	VARCHAR(500)	Ruta a clave privada SSH
is_local	TINYINT(1)	1 si es el mismo servidor del panel
status	TINYINT(1)	1=activo
date_creation	DATETIME	Fecha creación

llx_deploymanager_instance

Campo	Tipo	Descripción
rowid	INT AI PK	ID
fk_server	INT FK	Servidor donde está
name	VARCHAR(150)	Nombre
domain	VARCHAR(255)	Dominio (ej: erp.micliente.com)

Campo	Tipo	Descripción
custom_path	VARCHAR(500)	Ruta absoluta a htdocs/custom
conf_path	VARCHAR(500)	Ruta absoluta a conf/conf.php
environment	VARCHAR(20)	production/staging/development
status	TINYINT(1)	1=activo
note_public	TEXT	Notas
date_creation	DATETIME	Fecha creación

llx_deploymanager_module

Campo	Tipo	Descripción
rowid	INT AI PK	ID
slug	VARCHAR(100) UNIQUE	Identificador (nombre directorio)
display_name	VARCHAR(150)	Nombre para mostrar
has_migrations	TINYINT(1)	Si tiene migraciones SQL
date_creation	DATETIME	Fecha creación

llx_deploymanager_release

Campo	Tipo	Descripción
rowid	INT AI PK	ID
fk_module	INT FK	Módulo
version	VARCHAR(20)	Versión (ej: 2.0.0)
zip_path	VARCHAR(500)	Ruta al ZIP almacenado
zip_hash	VARCHAR(64)	Hash SHA-256 del ZIP
changelog	TEXT	Notas de cambios
fk_user_author	INT FK	Usuario que subió
date_creation	DATETIME	Fecha subida
UK: (fk_module, version)		Una versión por módulo

llx_deploymanager_instance_module

Campo	Tipo	Descripción
rowid	INT AI PK	ID
fk_instance	INT FK	Instancia
fk_module	INT FK	Módulo
installed_version	VARCHAR(20)	Versión instalada
last_scan	DATETIME	Último escaneo
UK: (fk_instance, fk_module)		Un registro por combinación

llx_deploymanager_batch

Campo	Tipo	Descripción
rowid	INT AI PK	ID
fk_module	INT FK	Módulo desplegado
fk_source_instance	INT FK	Instancia origen
description	VARCHAR(255)	Descripción auto-generada
total_count	INT	Total instancias
completed_count	INT	Completadas
failed_count	INT	Fallidas
status	VARCHAR(20)	running/completed/partial_failure/failed
fk_user_author	INT FK	Usuario que lanzó
date_creation	DATETIME	Inicio
date_completion	DATETIME	Fin

llx_deploymanager_deployment

Campo	Tipo	Descripción
rowid	INT AI PK	ID
fk_batch	INT FK	Lote de despliegue
fk_instance	INT FK	Instancia destino
status	VARCHAR(20)	pending/backing_up/deploying/migrating/verifying/completed/failed
backup_path	VARCHAR(500)	Ruta al backup creado
date_start	DATETIME	Inicio
date_end	DATETIME	Fin
log	TEXT	Log completo del despliegue
error_message	TEXT	Error si falló

Clases PHP

SSHExecutor

Wrapper para ejecutar comandos SSH, rsync y SCP en servidores remotos.

```
$ssh = new SSHExecutor($serverObject);
```

Constructor: recibe un objeto con host, ssh_user, ssh_port, ssh_key_path, is_local.

Métodos:

Método	Descripción
<code>exec(\$command, &\$output, &\$exitCode)</code>	Ejecuta comando. Local: exec directo. Remoto: escribe comando en tmpfile, ejecuta vía ssh <code>bash < tmpfile</code>
<code>testConnection()</code>	Ejecuta <code>echo OK</code> y verifica respuesta
<code>rsync(\$localPath, \$remotePath, &\$output)</code>	Rsync con <code>-delete</code> . Valida que <code>remotePath</code> contenga <code>/htdocs/custom/</code> (solo remoto)
<code>backup(\$remotePath, \$moduleName, \$localBackupDir)</code>	Tar.gz remoto + SCP al panel
<code>getDbCredentials(\$confPath)</code>	Lee <code>conf.php</code> remoto con <code>php -r</code> , devuelve array con <code>host/name/user/pass/prefix</code>
<code>runMigration(\$confPath, \$sqlFilePath)</code>	Ejecuta SQL remoto usando archivo temporal de credenciales (no expone password en ps)
<code>setOwner(\$remotePath, \$owner)</code>	<code>chown -R</code>
<code>fileExists(\$remotePath)</code>	<code>test -e</code>
<code>readFile(\$remotePath)</code>	<code>cat</code>

Seguridad: todos los parámetros de shell usan `escapeshellarg()`. Conexiones SSH con `-o BatchMode=yes` (no pide password interactivo).

DeployEngine

Orquestador principal. Gestiona el ciclo completo de despliegue.

```
$engine = new DeployEngine($db);
```

Métodos públicos:

Método	Descripción
<code>findSourceInstance(\$moduleId)</code>	Busca la instancia con la versión más alta del módulo (orden numérico)
<code>createBatch(\$moduleId, \$sourceInstanceId, \$instanceIds, \$userId)</code>	Crea batch + deployments en BD
<code>executeBatch(\$batchId)</code>	Ejecuta todos los despliegues del batch secuencialmente
<code>getBatchStatus(\$batchId)</code>	Devuelve estado completo del batch con deployments

Flujo de `executeDeployment()`:

- BACKUP (status: `backing_up`)
 - SSHExecutor::backup() → tar.gz + SCP
- DEPLOY (status: `deploying`)
 - rsyncBetweenServers()

- └─ Mismo servidor: rsync -a --delete via SSH
- └─ Diferente servidor: descarga a /tmp/ del panel → sube al destino

3. GIT SYNC

- └─ SSHExecutor::exec()
 - └─ git fetch + reset --hard origin/main
 - └─ rsync de nuevo (reaplica cambios sobre el reset)
 - └─ git add + commit + push

4. SQL (status: migrating)

- └─ SSHExecutor::runMigration() para cada *.sql

5. VERIFY (status: verifying)

- └─ Lee versión del mod*.class.php remoto
- └─ Actualiza llx_deploymanager_instance_module

6. COMPLETE (status: completed | failed)

rsyncBetweenServers(): validación de rutas: - Rechaza paths con .. - Rechaza paths sin /custom/ - Cross-server: descarga del origen al panel (/tmp/dm_rsync_*/), sube al destino, limpia temp

ModuleScanner

Descubre módulos instalados en una instancia vía SSH.

```
$scanner = new ModuleScanner($db);  
$result = $scanner->scanInstance($instance, $server);
```

Ejecuta un script bash que: 1. Lista directorios en custom_path/*/ 2. Busca core/modules/mod*.class.php en cada uno 3. Extrae versión con grep (fallback a version.txt) 4. Devuelve slug|version por línea

Resultado: upsert en llx_deploymanager_instance_module y auto-registro de módulos nuevos en llx_deploymanager_module.

ModulePackager

Valida y almacena ZIPs de releases.

```
$packager = new ModulePackager($db);  
$result = $packager->processUpload($tmpFile, $originalName,  
    $userId);
```

Flujo: 1. Extrae ZIP en directorio temporal 2. Valida seguridad: sin .., sin rutas absolutas, sin ejecutables (.sh, .py, .phar, .so, .exe, .bat) 3. Busca core/modules/mod*.class.php para identificar slug y versión 4. Verifica que no exista esa versión 5. Crea ZIP limpio en data/releases/[slug]/[version].zip 6. Calcula hash SHA-256 7. Inserta en llx_deploymanager_release

Endpoints AJAX

POST ajax/deploy_execute.php

Lanza un despliegue. Ejecuta en background tras enviar respuesta (fastcgi_finish_request).

Permiso: deploymanager->deploy

Body (JSON):

```
{
  "modules": [
    {"module_id": 5, "source_instance_id": 12}
  ],
  "instance_ids": [3, 7, 15, 22]
}
```

Respuesta:

```
{"ok": true, "batch_ids": [42]}
```

GET ajax/deploy_status.php?id={batchId}

Consulta estado de un batch.

Permiso: deploymanager->leer

Respuesta: objeto completo con batch, módulo, instancia origen y array de deployments con status/log.

POST ajax/scan_instance.php?id={instanceId}

Escanea módulos de una instancia.

Permiso: deploymanager->leer

Respuesta:

```
{"ok": true, "modules": [{"slug": "mimodulo", "version": "2.0.0"}, ...]}
```

GET ajax/test_connection.php?id={serverId}

Prueba conexión SSH a un servidor.

Permiso: deploymanager->admin

Respuesta:

```
{"ok": true, "message": "Conexión SSH correcta"}
```


POST ajax/upload_release.php

Sube un ZIP de módulo (multipart/form-data).

Permiso: deploymanager->deploy

Form fields: module_zip (file), changelog (text opcional)

Respuesta:

```
{"ok": true, "release_id": 15, "module_slug": "mimodulo",  
  "version": "2.1.0", "zip_hash": "abc123..."}
```

GET ajax/autodiscover.php?server_id={id}

Descubre instancias Dolibarr en un servidor buscando archivos conf/conf.php.

Permiso: deploymanager->admin

Respuesta:

```
{"ok": true, "added": 5, "skipped": 12, "instances":  
  ["erp.cliente1.com", "erp.cliente2.com"]}
```

Sistema de permisos

Definidos en modDeployManager.class.php:

ID	Permiso	Descripción
500501	deploymanager->leer	Ver dashboard, instancias, módulos
500502	deploymanager->deploy	Ejecutar despliegues, subir releases
500503	deploymanager->admin	CRUD servidores e instancias

Todos los endpoints AJAX y páginas verifican permisos al inicio:

```
if (!$user->admin && empty($user->rights->deploymanager->deploy))  
{  
    accessforbidden(); // o http_response_code(403) en AJAX  
}
```

Configuración

Constantes Dolibarr

Constante	Default	Descripción
DEPLOYMANAGER_DATA_PATH	DOL_DATA_ROOT/ deploymanager	Ruta para releases, backups, tmp
DEPLOYMANAGER_SSH_KEY	(vacío)	Ruta a clave SSH para cross-server

Clave SSH

Para despliegues entre servidores, se usa una clave SSH dedicada:

```
# Generar (una vez, en el servidor del panel)
ssh-keygen -t ed25519 -f /ruta/dolibarr/data/deploymanager_key -N ""
```

```
# Copiar a cada servidor destino
ssh-copy-id -i /ruta/dolibarr/data/deploymanager_key.pub
    deployer@192.168.1.10
ssh-copy-id -i /ruta/dolibarr/data/deploymanager_key.pub
    deployer@192.168.1.20
```

Estructura de datos en disco

```
data/deploymanager/
├── releases/
│   ├── mimodulo/
│   │   ├── 2.0.0.zip
│   │   └── 2.1.0.zip
│   └── otromodulo/
│       └── 1.0.5.zip
├── backups/
│   ├── erp.cliente1.com/
│   │   └── mimodulo_20260507_143022.tar.gz
│   ├── erp.cliente2.com/
│   │   └── mimodulo_20260507_143025.tar.gz
├── tmp/
│   └── upload_6645a3b2e1f4a/ (temporales, se borran)
```

Seguridad

Medidas implementadas

SQL Injection: todas las queries usan `$db->escape()` para strings y `(int)` casting para enteros.

Command Injection: todos los parámetros de shell usan `escapeshellarg()`. Los comandos complejos se escriben en un archivo temporal y se ejecutan vía `bash < tmpfile` (evita problemas de escaping de comillas).

Path Traversal: `rsyncBetweenServers()` rechaza rutas con `..` y rutas que no contengan `/custom/`.

XSS: todo el output HTML usa `do_escape_htmltag()`.

Autenticación: todos los endpoints verifican sesión Dolibarr y permisos específicos del módulo.

CSRF: las páginas UI verifican `GETPOST('token') == $_SESSION['newtoken']` en operaciones de escritura. Los endpoints AJAX de deploy y upload usan `NOCSRFCheck` porque reciben JSON/multipart, no formularios estándar.

Credenciales BD: nunca se almacenan. Se leen en tiempo real del `conf.php` remoto y se usan vía archivo temporal con `chmod 600` (no aparecen en `ps aux`).

ZIP Upload: se valida estructura del módulo, se rechazan ejecutables y rutas peligrosas, se calcula hash SHA-256.

Superficie de ataque

El módulo tiene acceso SSH a todos los servidores configurados. La seguridad depende de: 1. Acceso al panel de Dolibarr (restringido por IP en `.htaccess` + login) 2. Permisos del módulo (solo admins pueden desplegar) 3. Clave SSH dedicada (revocar una clave revoca todo el acceso)

Cómo añadir un nuevo servidor

1. **En el servidor destino:** crear usuario deployer o usar root
2. **Copiar clave SSH:** `ssh-copy-id -i deploymanager_key.pub deployer@ip-del-servidor`
3. **En el panel:** Servidores → Añadir → rellenar datos → Test conexión
4. **Autodiscover:** detectar instancias automáticamente
5. **Escanear:** escanear módulos de cada instancia

Cross-server rsync

Cuando origen y destino están en servidores diferentes:

Servidor Origen —(rsync download)→ Panel (/tmp/dm_rsync_*/)
—(rsync upload)→ Servidor Destino

1. El panel descarga el módulo del servidor origen a un directorio temporal local
2. El panel sube ese directorio al servidor destino
3. Se limpia el directorio temporal

Para servidores iguales, se ejecuta un rsync local directamente vía SSH.